Full Stack Development With MERN Stack

1. Introduction

- **Project Title: Customer Registry**

- **Description:**

The **Customer Registry System** is a web-based application developed to manage customer information and complaint handling efficiently. It allows customers to register, log in, create and manage their profiles, and raise complaints. Administrators can manage customer records, assign complaints to agents, and monitor system activities, while agents can view assigned complaints, communicate with customers, and update complaint statuses. The system provides role-based access, secure authentication, and an easy-to-use interface for effective customer relationship management.

- **Team Members:**

Name	Role
Bathina Pravallika	Reporting (Team lead)
Gunupudi Surya Swarnitha	Front end Developer
Gullipalli Haritha	Back end Developer
Srigiri sree Pallavi	Documentation
Ankamreddy Parvathi Prathiba	Tester

---

2. Project Overview

- **Purpose:**

The purpose of the **Customer Care Registry** is to provide a centralized platform for managing customer complaints, service requests, and interactions efficiently. The application enables users to register complaints, track their status, and communicate with support agents, while allowing agents to manage and resolve issues effectively. By maintaining organized customer records and providing timely support, the system improves service quality, enhances customer satisfaction, and streamlines the overall customer support process.

- **Features:**

  - **Customer Profile Creation**

Create customer profiles with essential details like name, contact, and address.

○ □ **Contact Management**

Add, edit, and delete customer contact details, including phone numbers and email addresses.

○ □ **Customizable Fields**

Create custom fields to store business-specific customer information.

○ □ **User Registration**

Register customers, admins, and agents with secure login credentials.

○ □ **Admin Dashboard**

Manage customer data, user accounts, and system settings from a centralized dashboard.

○ □□ **Agent Accounts**

Create agent profiles and provide access to customer information for support.

○ □ **Customer Information Management**

View and update customer details, addresses, and communication preferences.

○ □ **Communication Channels**

Enable email and notification-based communication between customers, admins, and agents.

○ □ **Communication History**

Track emails, calls, meetings, and support interactions for better customer management.

---

3. Architecture

- **Frontend (React.js):**

The frontend provides the **user interface** for customers ,agents, and administrators to interact with Customer Registry System.

- **Technology Used:** React.js/Vite/JavaScript(ES6+)/HTML5/CSS3/React Router
- **Features:**
  - Login / Register Screen
  - Customer Profile Management
  - Customer List Display
  - Add, Edit & Delete Customer Records
  - Search & Filter Customers

        - Admin & Agent Dashboard
        - Responsive User Interface

- **Backend (Node.js & Express.js):**

  The backend manages business logic, authentication, and customer-related operations.

    - **Technology Used:**Node.js / Express.js / JWT / bcrypt.js / REST AP
    - **Functionality:**
        - User Authentication & Authorization
        - Customer CRUD Operations
        - Role-Based Access Control
        - API Request Handling
        - Input Validation & Error Handling

- **Database (MongoDB):**

- **Technology Used:** MongoDB

- **Purpose:**

    - Store user login details
    - Store Customer Profiles
    - Store Contact Information
    - Manage User Roles (Admin, Agent, Customer)
    - Store Customer Records Securely

---

4. Setup Instructions

- **Prerequisites:**

    - **Software Requirements:**

        - Node.js (v16 or above)
        - npm (Node Package Manager
        - Git
        - Visual Studio Code
        - MongoDB (Local or MongoDB Atlas)

    - **Frontend Technologies:**

        - React.js – for building the user interface
        - Vite – for fast development and build process
        - JavaScript (ES6+) – for frontend logic
        - HTML5 & CSS3 – for page structure and styling

        - React Router DOM – for page navigation
        - Axios – for API communication
    - **Hardware Requirements:**
        - Minimum: 4 GB RAM (for basic testing)
        - Recommended: 8 GB RAM
        - Internet Connection – for package installation and database access
    - **Dataset Requirements:**
        - MongoDB – for storing customer and user data
        - Mongoose – for database modeling (if implemented)
    - **Backend Technologies:**
        - Node.js – for server-side runtime
        - Express.js – for building REST APIs
        - JWT – for user authentication (if implemented)
        - bcrypt.js – for password encryption (if implemented)

- ## Installation Steps:

npm install

npm install react-router-dom

npm install axios

npm install --save-dev nodemon

npm install

npm install express

npm install mongoose

npm install cors

npm install dotenv

npm install jsonwebtoken

npm install bcryptjs

npm install axios

---

5. Folder Structure:

- **Client (React Frontend):**
  - public/ – Stores static assets.
  - assets/ – Contains images, icons, and other resources.
  - components/ – Reusable React components.
  - pages/ – Application pages such as Login, Register, Dashboard, and Customers.
  - services/ – Handles API communication.
  - App.jsx – Main React component.
  - main.jsx – Entry point of the application.
  - index.css – Contains global styles for the application.
- **Server (Node.js Backend):**
  - config/ – Stores database configuration files.
  - controllers/ – Contains business logic.
  - models/ – Defines MongoDB database schemas.
  - routes/ – Manages API routes and endpoints.
  - middleware/ – Handles authentication and request validation.
  - server.js – Entry point of the backend server.
  - .env – Stores environment variables and configuration settings.

---

6. User Interface

- **Login/Register Screen**
  - Allows users to register and log in securely.
- **Customer Dashboard**
  - Displays customer details and recent activities.
- **Customer Management**
  - Add, view, update, and delete customer records.
- **Complaint Management**
  - Raise complaints and track their status.
- **Admin Dashboard**
  - Manage customers, agents, and system activities.
- **Agent Dashboard**

○ View assigned complaints and update their status.

• **Notifications**

○ Displays updates and important system notifications.

---

7. Testing

• **Tools Used:**

o React.js – Frontend development and testing
o Node.js & Express.js – Backend API testing
o MongoDB – Database testing and validation
o Postman – API request and response testing
o Visual Studio Code – Development and debugging
o Google Chrome – User Interface testing

• **Strategy:**

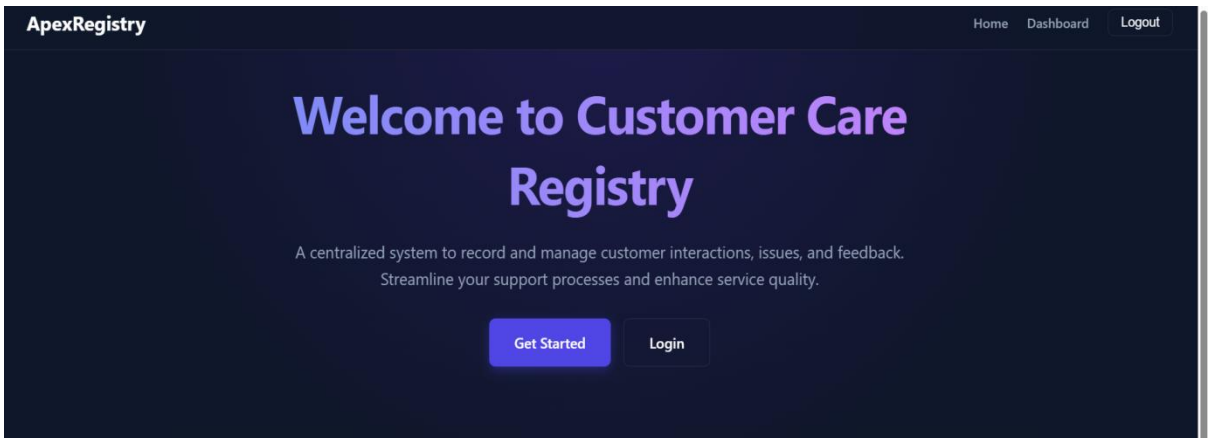
o Unit testing for frontend and backend functions.
o Integration testing for REST APIs and database operations.
o Functional testing for Login, Registration, Customer Management, and Complaint Management.
o User Interface testing to verify navigation and responsiveness.
o End-to-end testing to ensure smooth user workflows.

8.Screenshots or Demo

- □ Screenshots of Key Pages:

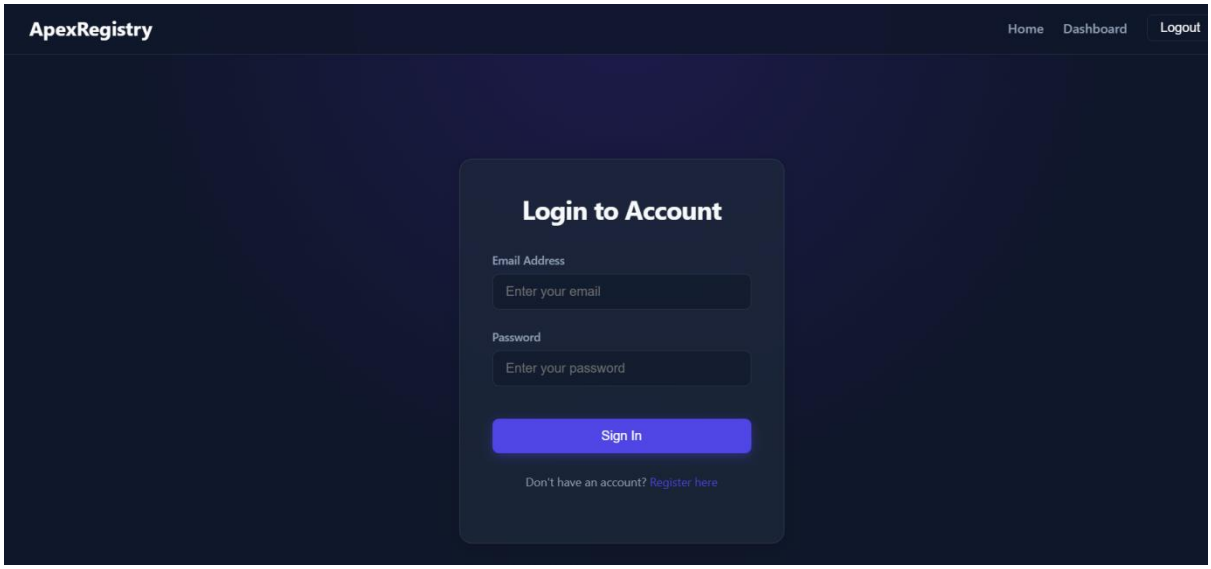
• **User Home Page**

○ This is the Home Page of the Customer Registry application.

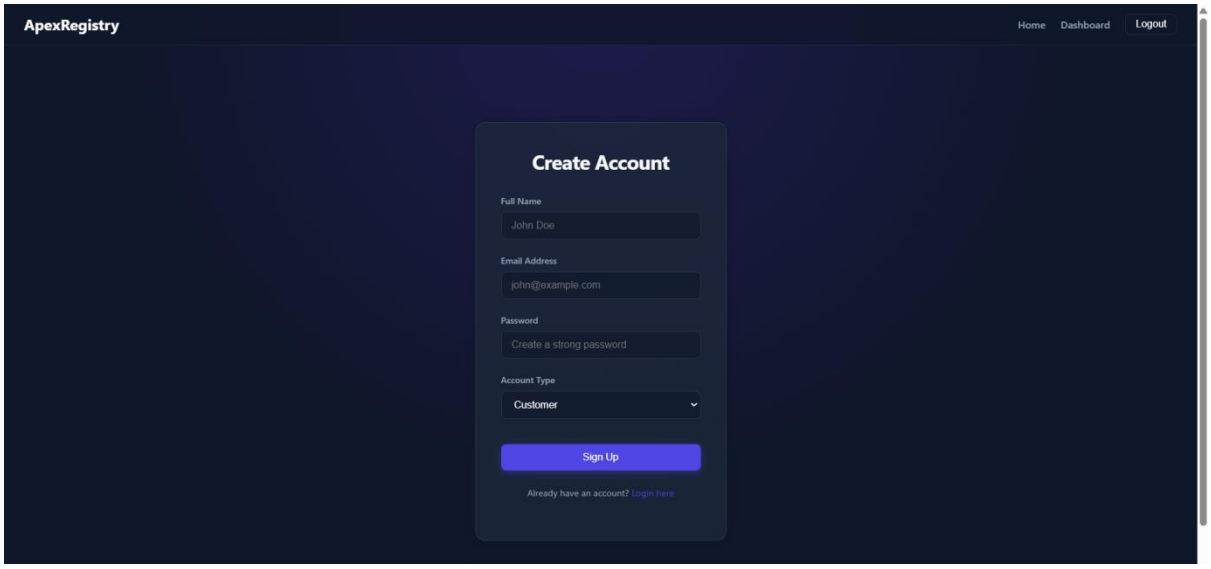


• **Login Page**

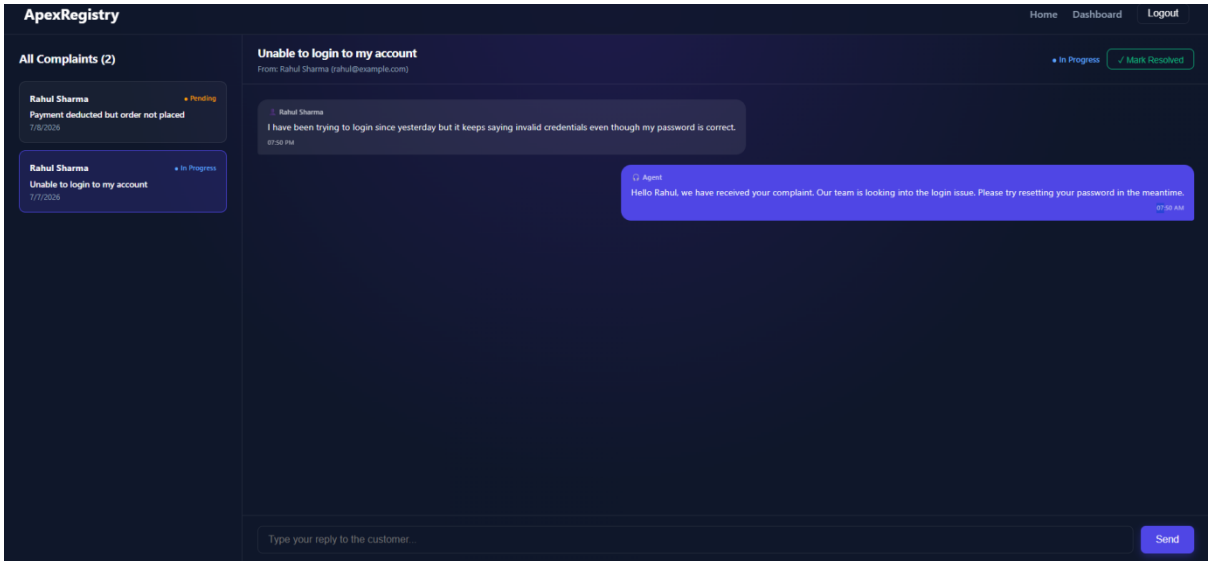
○ This page allows registered users to securely log in.



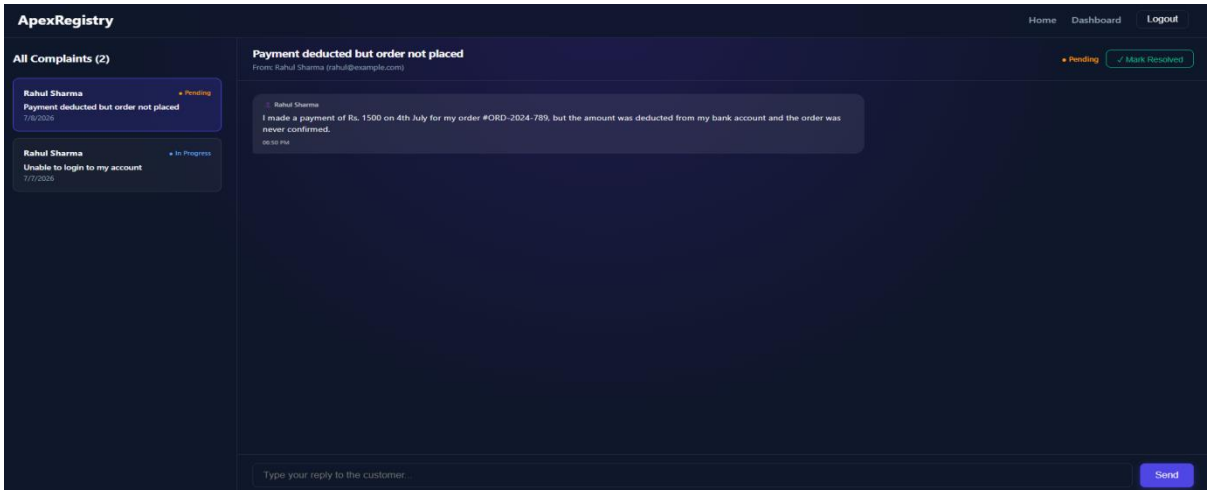
- **Register Page**
  - This page enables new users to create an account.



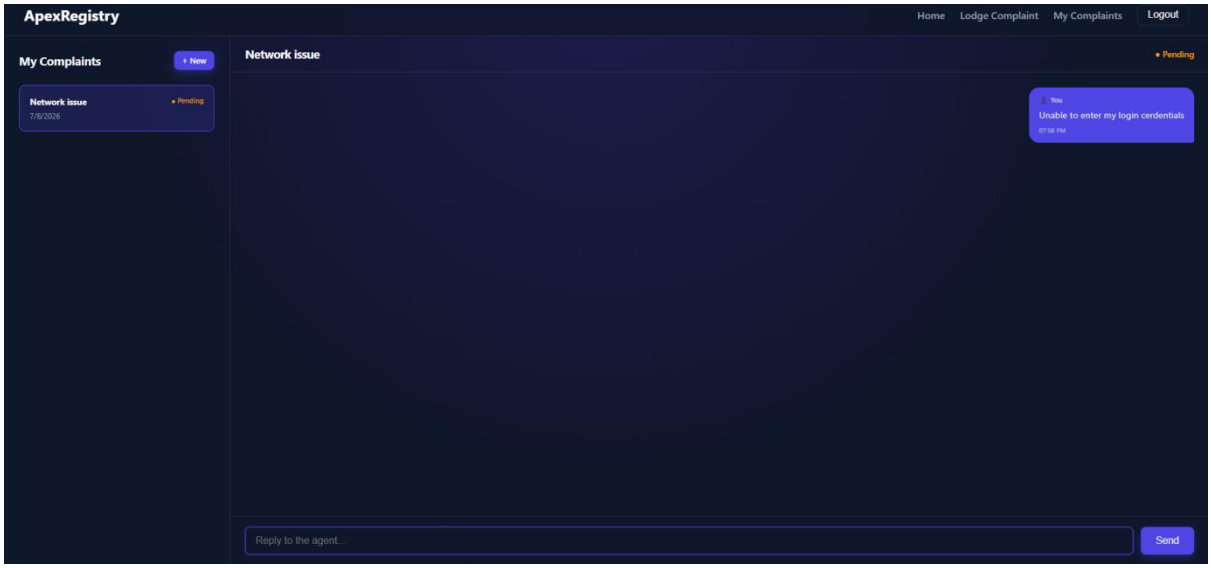
- **Agent Dashboard**
  - This dashboard allows agents to manage customer complaints and provide support.



- **My Complaints Page**
  - This page allows customers to view and track their registered complaints.



- **Complaint Management Dashboard**
  - This dashboard enables agents or administrators to manage and resolve customer complaints.



9.Known Issues

1. □ **Internet Connection Required**
   The application requires a stable internet connection to access backend services.

2. □ **Role-Based Access Restrictions**
   Users can only access features based on their assigned role (Admin, Agent, or Customer).

3. □ **Complaint Response Delay**
   Complaint updates depend on agent availability and may not be updated instantly.

4. □ **Limited Mobile Optimization**
   Some pages may not display perfectly on smaller mobile devices.

---

10. Future Enhancements:

1. □ **Mobile Application Support**

   Develop a mobile app for easy access.

2. □ **Cloud Deployment**

   Deploy the system for better accessibility.

3. □ **Email & SMS Notifications**

   Send instant complaint status updates.

4. □ **Live Chat Support**

   Enable real-time communication.

5. □ **Reports & Analytics**

   Generate reports and system insights.

11. APPENDIX:

GitHub_Repository_Link : https://github.com/GSURYASWARNITHA1/Customer-Registry_Smart

Project Demo: https://customer-registry-frontend.onrender.com